

COSC 39: Theory of Computation

Problem 1. No fast automata checker.

Consider the following problem about NFAs.

NFA-REJECT

- **Input:** An n -state NFA N .
- **Output:** Does NFA N reject at least one string? In other words, is $L(N) \neq \Sigma^*$?

Prove that the NFA-REJECT problem is NP-hard.

[We have demonstrated in class that analyzing the behavior of a program in general is undecidable; here we see that even when the program has no access to memory we cannot expect to have efficient algorithms!]

Solution.

To prove that the problem NFA-REJECT is NP-Hard, we can demonstrate that it is at least as hard as any problem in NP. We achieve this by a polynomial time reduction from an already known NP-Hard problem to NFA-REJECT. This reduction demonstrates that for any input of our NP-hard problem, we can convert it to an input of NFA-REJECT and solve that, then convert it back to a solution for our NP-Hard problem. Thus, NFA-REJECT must be at least as hard as any NP-Hard problem, proving its NP-Hard.

For our reduction, let problem A be the 3-SAT problem and problem B be the NFA-REJECT problem. We will prove the reduction:

$$A \leq B$$

To begin, start with a generalized set up of the 3-SAT problem. For example:

$$(x \vee \bar{y} \vee w) \wedge (\bar{x} \vee y \vee \bar{z})$$

Assume we have n variables and m clauses, with each clause having 3 literals. Our goal is to construct an NFA N such that it only rejects at least one string *if and only if* the original 3-SAT has a satisfiable solution. Construct an NFA N as follows:

First, arbitrarily decide an ordered sequence of the variables—any permutation is sufficient. For our example, we will have our order be x, y, z, w . Then proceed with the following construction:

Define the alphabet $\Sigma = \{0, 1\}$

Make a length enforcement gadget. It should accept all strings w such that $|w| \neq n$, where n is the number of variables. Now we can interpret the string w as an assignment of $\{0, 1\}$ to each variable. Construct it as follows:

1. Construct a start accepting state q_0
2. Create an edge with transition value Σ to a new accepting state q_i . Repeat this for a total of $1 < i < n - 1$ times.
3. For the n th state in the chain, make it a reject state.
4. Lastly, make one for edge with transition value Σ to a new trap accepting state such that this accepting state also has a self-loop with edge transition Σ

This gadget rejects string input w **if and only if** the length of $|w| = n$.

Right Direction $\rightarrow$: Assume that the gadget rejects some input string w . The gadget has no fall-off opportunities or reject states other than 1 state, state q_n . Since there is only one path to state q_n , follow that path. There are exactly n edges between q_0 (the start state) and q_n , meaning the input string w must have length n .

Left Direction $\leftarrow$: Assume that the length of input string $|w| = n$. Starting from the start state, follow the only available path of Σ transitions for the length of w . Since there are exactly n transitions after the start state, following input string w will yield final state q_n , which is a reject state.

That proves that the gadget only rejects strings of length n , where n represents the number of variables. This will be crucial for our reduction as we convert our n variable assignment input to our 3-SAT to the n length string input to our NFA N .

An example of the NFA gadget is below for $n = 4$



Now, construct the set of gadgets that ensures NFA N rejects at least 1 input string **if and only if** there is a satisfiable assignment to our 3-SAT problem.

For each clause m comprised of 3 of the following variables x, y, z, w , build the following “gadget”.

1. If there exists a variable and its compliment— x and $\bar{x}$ for example—within the same clause, then simply do not make the gadget. This clause will always evaluate to true.

Thus, they can be dropped from the preprocessing step before construction, since they are trivially satisfied and do not affect satisfiability.

2. Otherwise, do the following:
3. Create a state for every variable (including the variables not in the clause) in a downward column in the order of the pre-defined sequence. The first variable in the pre-defined sequence is the start state.
4. Create a dedicated trap reject state
5. Create a dedicated trap accept state
6. Iterate through the variables based on the ordered sequence.
7. If the variable is included in the clause, create a 1 transition to the trap state and a 0 transition to the next variable state in the sequence. The jump to the trap state represents identifying a true-valued literal and accepting the clause regardless of the other variables. The jump to the next variable represents the literal not evaluating to true and thus checking the next variable state.
8. If the counterpart, or barred version of the variable is included in the clause, create a 0 transition to the trap state and a 1 transition to the next variable state in the sequence. This is the same logic as the previous point just with the value flipped.
9. For the last variable in the sequence, in our example w , rather than making an edge to the next variable, make that edge to the trap accept state, which serves as the final state in the chain.
10. have self loops (with value alphabet) for the trap reject state and trap accept state

That Completes our gadget logic.

Below is an example of the gadget which is constructed for the clause:

$$(x \vee \bar{y} \vee w)$$



I claim that the NFA N rejects an input **if and only if** the 3-SAT has a satisfiable assignment.

$$\varphi \text{ is satisfiable} \Leftrightarrow L(N) \neq \Sigma^*$$

Right Direction $\rightarrow$: Assume that we have some satisfiable assignment to φ . Fix input w as the sequence of 0s and 1s such that the i th variable in the defined sequence has the i th value in w . We argue that the constructed NFA N rejects string w .

First, the length gadget: Since $w = n$, the length gadget path ends in state q_n , which is a reject state.

Next, the clause gadgets: Since φ is satisfiable, that must mean that for **every** clause m_i in the 3-SAT, *some* literal has an assignment of 1. By the construction of the clause gadgets, when the mentioned j th variable satisfies the clause m_i , the j th character in input string w transitions from the j th variable's state to the trap state, where it remains indefinitely. Thus, every clause gadget ends up in a reject state in addition to the length gadget. Since every path/finger of input string w ends up in a reject state, that must mean $L(N) \neq \Sigma^*$.

Left Direction $\leftarrow$: Assume that there exists some string w that NFA N rejects. We argue that we have some satisfiable assignment to φ .

First, the length gadget: Since NFA N rejects, it must have ended at state q_n , meaning $|w| = n$. Interpret w as a sequence of 0s and 1s such that the i th variable in the defined sequence has the i th value in w .

Since N rejects w , every path on w ends in a reject state. For each clause gadget, m_i , the path through the gadget must end up in the trap reject state.

By construction, the only way a clause gadget path ends in the trap state is if some variable state s_j caused the path to go there, which only happens when the j th variable (which is in the clause) is satisfied by the j th character in w . If no literal in the clause m_i were satisfied by w , then the path would pass through every variable state without going to the trap reject state, and end up in the trap accept state, which would contradict that every path rejects.

Thus, for every clause m_i , at least one literal is satisfied by w , meaning w satisfies φ , proving φ is satisfiable.

Now that we have proven the reduction, including both directions, now we just need to prove that the reduction can be done in polynomial time.

1. The length gadget has $O(n)$ states
2. Each clause gadget has $O(n)$ states ($n +$ the 2 trap states)
3. There are m clauses, so there are $O(mn)$ states
4. Additionally, we need to check every possible pair in each clause to see if there is a variable x and its compliment $\bar{x}$. Since each clause has only 3 literals max, we can check this in $O(1)$ time. Since there are m clauses total, this check is $O(m)$ time.

Since all of these processes occur in polynomial time, we can do our reduction in polynomial time.

Problem 2. Unfinished coursework.

A *dominating set* in graph G is a set of vertices S so that every vertex is in S or a neighbor of S .

DOMSET

- **Input:** An undirected graph G , a parameter q
- **Output:** Does G have a dominating set of size at most q ?

Prove that DOMSET does not have sub-exponential time algorithm under ETH.

Problem 3. Many puzzles, board games, and video games are NP-hard; the list is so long that I cannot even. (See this course webpage of Erik Demaine.) Most signature examples includes Baba is You (actually undecidable), Candy Crush Saga, Chess, Cookie Clicker, Doom, Go, Grand Theft Auto, Legend of Zelda, Magic the Gathering (actually undecidable), Minesweeper, Pokémon, Portal, Resident Evil, Rush Hour, Sudoku, Super Mario Bros., Tetris, and more.

Prove one of your favorite games is NP-hard. (To get full credit, you need to choose a game that has not been proven to be NP-hard before.)

Solution.

We reduce a sparse k -SAT instance to a DOMSET instance in the following manner:

Let $q = n / \log n$. Create an undirected graph G as follows:

1. Create q groups $X_1, \dots, X_q$ of n variables. Create a node X_{ij} for each possible assignment of a group X_i . That gives us $q2^{n/q}$ nodes.

Also create a node n_s for each clause ($1 \leq s \leq m$). Hence we have a total of $q2^{n/q} + m$ nodes.

2. Every X_i is a complete subgraph. If there exists a node X_{ij} that satisfies some clause n_s create an edge between them.

We claim that G has a dominating set of size q if and only if the k -SAT instance is satisfiable.

($\implies$)

Let S be a dominating set with $|S| = q$. Then S contains exactly one node from each group X_i since for any node $u \in X_i$ whose assignment satisfies no clause, u has no edges to any clause node. Then u 's only neighbors are other nodes in X_i , so u can only be dominated if $S \cap X_i \neq \emptyset$. Since this holds for every group and there are q groups with $|S| = q$, we

conclude $|S \cap X_i| = 1$ for each i . (If two nodes in X_i satisfy a clause we just pick one since those nodes are neighbors anyway).

($\Leftarrow$)

Suppose σ is a satisfying assignment. For each group X_i , let v_i be the node corresponding to σ 's assignment on X_i 's variables, and let $S = \{v_1, \dots, v_q\}$. We claim S is a dominating set of size q :

- Every node $u \in X_i$ is dominated by v_i , since X_i is a complete subgraph and thus v_i is adjacent to every other node in X_i .
- Every clause node n_s is dominated: since σ is satisfying, some group X_i contains a variable that satisfies clause s . Then v_i , which agrees with σ on X_i , also satisfies s , so the edge $v_i \sim n_s$ exists by construction.

Thus we have reduced sparse k -SAT to DOMSET. Now we show that if DOMSET having an algorithm that solves it in $O(n^{o(q)})$ time, where n is the number of nodes in the graph, by contradiction. Assume such an algorithm exists. Then,

$$O(n^{o(q)}) = O\left(\left(q2^{n/q} + m\right)^{o(q)}\right) = O\left(\left(\frac{n}{\log n}2^{\log n} + m\right)^{o\left(\frac{n}{\log n}\right)}\right)$$

By the sparsification lemma we have $m = \Theta(n) = \alpha n$ for some α . Also note that $2^{\log n} = n$, so

$$O(n^{o(q)}) = O\left(\left(\frac{n}{\log n}n + \alpha n\right)^{o\left(\frac{n}{\log n}\right)}\right) = O\left(\left(\frac{n^2}{\log n} + \alpha n\right)^{o\left(\frac{n}{\log n}\right)}\right)$$

Note that $\frac{n^2}{\log n}$ clearly dominates, so we drop the other term.

$$\begin{aligned} O(n^{o(q)}) &\leq O\left(\left(\frac{n^2}{\log n}\right)^{o\left(\frac{n}{\log n}\right)}\right) = O\left(\left(2^{\log(n^2/\log n)}\right)^{o\left(\frac{n}{\log n}\right)}\right) \\ &= O\left(\left(2^{2\log(n) - \log \log n}\right)^{o\left(\frac{n}{\log n}\right)}\right) \\ &\leq O\left(\left(2^{O(\log n)}\right)^{o\left(\frac{n}{\log n}\right)}\right) = O\left(2^{O(\log n) \cdot o\left(\frac{n}{\log n}\right)}\right) = O\left(2^{o(n)}\right) \end{aligned}$$

This gives us an $O(2^{o(n)})$ algorithm (which is strictly less than $O(2^n)$) that solves k -SAT, contradicting ETH. Thus we cannot solve DOMSET in $O(n^{o(q)})$ time.